📦 AWS Core Fundamentals — Category 1: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Intermediate
> **Topics Covered:** Global Infrastructure, IAM, CLI & SDK, Organizations & Multi-account Strategy

---

📋 Table of Contents

1. 1.1 AWS Global Infrastructure — Regions, AZs, Edge Locations
2. 1.2 IAM — Users, Groups, Roles, Policies
3. 1.3 AWS CLI & SDK Basics
4. 1.4 AWS Organizations & Multi-account Strategy

1.1 AWS Global Infrastructure — Regions, AZs, Edge Locations

🟢 What It Is in Simple Terms

AWS runs physical data centers around the world, organized into a three-tier geographic hierarchy: Regions (large geographic areas), Availability Zones (individual data centers within a region), and Edge Locations (lightweight cache/compute nodes closer to end users). Understanding this hierarchy tells you where your data lives, how resilient your architecture is, and what the latency profile looks like for your users.

🔍 Why It Exists / What Problem It Solves

```
Problems the global infrastructure solves:
├── Latency:       serve users from a data center near them
├── Compliance:    "data must not leave EU" → deploy in eu-west-1
├── Resilience:    one data center burns down → other AZs absorb traffic
├── Blast radius:  a bug in one AZ shouldn't take down your entire service
└── Redundancy:    power, cooling, networking — all independent per AZ
```

🧩 Regions

```
Region = a named geographic area containing multiple AZs
         Completely independent — no shared power, networking, or control
plane
```

```
Examples:
├── us-east-1       (N. Virginia — AWS's largest, oldest region)
├── us-west-2       (Oregon)
├── eu-west-1       (Ireland)
├── ap-southeast-1  (Singapore)
└── ap-south-1      (Mumbai)

As of 2025: 33+ regions globally, with more announced.

Region selection criteria (in order of priority):
1. Compliance / Data residency: GDPR → EU region. India DPDP → ap-south-1
2. Latency: where are your users? Use CloudFront + latency-based Route 53
3. Service availability: not all AWS services in all regions (e.g.,
Bedrock, Local Zones)
4. Cost: us-east-1 is typically cheapest. ap-southeast-1 is ~15% more
expensive
5. Proximity to other systems: same region as your partners / payment
processors

Key fact: Regions are COMPLETELY ISOLATED from each other by default.
          An EC2 instance in us-east-1 CANNOT talk to RDS in eu-west-1
          without explicit network configuration (VPC Peering, TGW,
internet).
```

🧩 Availability Zones (AZs)

```
Availability Zone = one or more physical data centers within a Region
                    connected to each other via low-latency private fiber

Each AZ:
├── Separate power grid (different utility substations)
├── Separate cooling systems
├── Separate physical security
├── Connected to other AZs in same region via <2ms round-trip latency
└── Named: us-east-1a, us-east-1b, us-east-1c, us-east-1d, us-east-1e, us-
east-1f

us-east-1 has 6 AZs — most regions have 3.
Minimum recommended: deploy across 3 AZs for production HA.

⚠ AZ naming is ACCOUNT-SPECIFIC:
   Your us-east-1a ≠ my us-east-1a (AWS maps AZ names differently per
account)
   This prevents everyone from landing in the same "a" AZ.
   Use AZ IDs (use1-az1, use1-az2) for cross-account AZ alignment.

Multi-AZ architecture pattern:
┌──────────────────────────────────────────────────────────┐
│                      us-east-1                           │
```

```
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │
│  │   AZ-a       │ │   AZ-b       │ │   AZ-c       │ │
│  │  EC2 (web)   │ │  EC2 (web)   │ │ EC2 (web)    │ │
│  │  EC2 (app)   │ │  EC2 (app)   │ │ EC2 (app)    │ │
│  │  RDS Primary │ │  RDS Standby │ │              │ │
│  │  Subnet-a    │ │  Subnet-b    │ │ Subnet-c     │ │
│  └──────────────┘ └──────────────┘ └──────────────┘ │
│       ALB distributes across all 3 AZs               │
└──────────────────────────────────────────────────────┘

If AZ-a fails: ALB automatically routes to AZ-b and AZ-c.
               RDS fails over to standby in AZ-b (<60 seconds).
               Application continues serving users — zero manual
intervention.
```

🧩 Edge Locations and the Content Delivery Tier

```
Edge Locations = CloudFront CDN nodes, Route 53 resolvers, WAF enforcement
points
                 400+ locations in 90+ cities globally

Purpose:
├── Cache static content (S3 objects, ALB responses) closer to users
├── Terminate TLS connections at the edge (reduce handshake latency)
├── Run Lambda@Edge / CloudFront Functions for edge compute
└── DNS resolution via Route 53 (always routed to nearest edge)

Regional Edge Caches:
├── Larger caches between edge locations and origin
└── Reduce cache misses that must go all the way back to us-east-1

Local Zones:
├── AWS infrastructure extensions into metro areas (e.g., Atlanta,
Chicago)
├── Latency < 10ms to the city (vs 60ms+ to nearest full region)
└── Used for: game servers, live video, financial trading, real-time
rendering

Wavelength Zones:
├── AWS infrastructure embedded inside telecom 5G networks
├── Single-digit millisecond latency to 5G mobile devices
└── Use case: autonomous vehicles, AR/VR, mobile gaming

Outposts:
├── AWS hardware rack shipped to YOUR data center
├── Run EC2, ECS, RDS, S3 on-premises with same APIs
└── Use for: data residency requirements, ultra-low latency to on-prem
systems
```

🧩 Global Infrastructure — How It Affects Architecture Decisions

```
Decision 1: Single-region vs Multi-region

Single-region + Multi-AZ:
├── Cost: baseline
├── RTO: ~1-5 minutes (AZ failover)
├── RPO: near-zero (synchronous replication within region)
├── Use for: most applications
└── Complexity: manageable

Multi-region Active-Passive:
├── Cost: ~1.5-2× (standby region with minimal resources)
├── RTO: 5-30 minutes (DNS failover + warm-up)
├── RPO: minutes (async cross-region replication lag)
└── Use for: disaster recovery, regulatory requirements

Multi-region Active-Active:
├── Cost: ~2× (full deployment in each region)
├── RTO: ~0 (automatic, Route 53 removes unhealthy region)
├── RPO: near-zero (DynamoDB Global Tables, Aurora Global DB)
└── Use for: global user base, sub-100ms latency worldwide, 99.999% SLA

Decision 2: Which region?
1. Compliance first (non-negotiable)
2. Where are your users? (check Route 53 latency routing or Cloudflare
analytics)
3. What services do you need? (check regional service availability)
4. What's the cost delta? (typically 5-20% between regions)
```

💬 Short Crisp Interview Answer

"AWS organizes infrastructure into three tiers. Regions are fully isolated geographic areas — us-east-1, eu-west-1 etc. — selected based on compliance requirements, user location, and service availability. Within a region, Availability Zones are separate physical data centers with independent power and networking, connected at under 2ms latency. The key architecture rule is: never deploy a production service in a single AZ — always span at least two, ideally three. Edge Locations are CloudFront CDN nodes, 400+ globally, that cache content and terminate TLS close to users. One gotcha: AZ names are account-specific — your us-east-1a may be a different physical data center than my us-east-1a. Use AZ IDs for cross-account coordination."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
AZ names are account-scoped	us-east-1a in your account ≠ us-east-1a in my account. Use AZ IDs (use1-az1)

Gotcha	Detail
Not all services in all regions	New services (Bedrock, some AI features) launch in us-east-1/us-west-2 first
Cross-region is NOT automatic	Data, traffic, and resources don't cross regions unless explicitly configured
Inter-AZ data transfer costs money	$0.01/GB between AZs within same region — design to minimize cross-AZ traffic
us-east-1 has more incidents	Oldest, largest region — some argue it has more historical incidents
Local Zones are not full regions	Limited service availability — no RDS Multi-AZ, no NAT Gateway in all Local Zones

---

1.2 IAM — Users, Groups, Roles, Policies

🟢 What It Is in Simple Terms

IAM (Identity and Access Management) is AWS's authorization system — it controls WHO can do WHAT to WHICH resources. Every API call to AWS goes through IAM. If IAM says no, nothing happens — no matter who you are or what you're trying to do.

🔍 Why It Exists / What Problem It Solves

```
Without IAM:
├── Anyone with AWS credentials = full access to everything
├── A compromised developer laptop = entire AWS account exposed
├── No audit trail of who did what
└── Services can't call other services without human credentials

With IAM:
├── Developers: read-only in prod, full access in dev
├── EC2 instances: can write to S3 but not delete it
├── Lambda: can query DynamoDB but cannot touch EC2
├── Cross-account: prod account resources accessible from CI/CD account
└── Every action: logged in CloudTrail with principal identity
```

🧩 IAM Identities

```
Four types of IAM identities:

1. IAM User:
```

```
    ├── Represents a PERSON or application needing long-term credentials
    ├── Has: username + password (console) + access keys (CLI/API)
    ├── Access keys = permanent until rotated or deleted
    └── ⚠ Best practice: don't create IAM users for applications.
                         Use IAM Roles instead (temporary credentials).

2. IAM Group:
    ├── Collection of IAM users — NOT an identity itself
    ├── Attach policies to group → all users in group inherit permissions
    ├── User can be in multiple groups
    └── Groups CANNOT be nested (no group inside a group)

3. IAM Role:
    ├── Identity WITHOUT long-term credentials
    ├── Assumed by: EC2, Lambda, ECS tasks, other AWS services,
    │               users (cross-account), SAML-federated identities, OIDC
    ├── Produces: temporary credentials (STS tokens, 15min–12hr lifetime)
    └── The RIGHT way to grant AWS services permissions to each other

4. IAM Identity Center (SSO) Users:
    ├── Modern replacement for IAM users for human access
    ├── Federated from: Okta, Azure AD, Google Workspace, etc.
    └── Single sign-on across multiple AWS accounts
```

🧩 IAM Policies

```
Policy = JSON document defining what is allowed or denied

Policy types:
├── Identity-based:   attached to user, group, or role
│   ├── AWS Managed:  pre-built by AWS (AmazonS3ReadOnlyAccess, etc.)
│   ├── Customer Managed: you create and manage
│   └── Inline:       embedded directly in the identity (avoid — hard to
manage)
│
├── Resource-based:   attached to a RESOURCE (S3 bucket, KMS key, SQS
queue)
│   └── Specifies WHO can access this resource (including cross-account
access)
│
├── Permission Boundaries: cap on what an identity policy can grant
│   └── Effective permissions = identity policy ∩ boundary
│
├── Service Control Policies (SCPs): org-level restrictions
│   └── Covered in 1.4 Organizations
│
└── Session policies: inline passed during AssumeRole
    └── Further restrict the role's permissions for that session

Policy structure:
```

```
{
  "Version": "2012-10-17",        ← always this date — it's a schema
version
  "Statement": [
    {
      "Sid": "AllowS3ReadProd",   ← optional statement ID for
documentation
      "Effect": "Allow",          ← Allow or Deny
      "Action": [                 ← what API calls
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [               ← which resources (ARN format)
        "arn:aws:s3:::prod-data-bucket",
        "arn:aws:s3:::prod-data-bucket/*"
      ],
      "Condition": {              ← optional: when does this apply?
        "StringEquals": {
          "s3:prefix": ["finance/", "reports/"]
        }
      }
    }
  ]
}
```

🧩 Policy Evaluation Logic — The Most Important IAM Concept

```
When an API call is made, IAM evaluates ALL applicable policies.
The evaluation order determines the final Allow or Deny:

Step 1: Explicit DENY — any policy has an explicit Deny?
        → DENY immediately. No exceptions. Full stop.

Step 2: SCPs — does the Organization SCP allow this action?
        → If SCP restricts it: DENY (even if identity policy allows)

Step 3: Resource-based policy — does the resource policy grant access?
        → Cross-account: resource policy alone CAN grant access
        → Same account: resource policy alone grants access

Step 4: IAM Permission Boundary — does the boundary permit this?
        → If boundary doesn't include this action: DENY

Step 5: Session Policy — does the session policy allow this?
        → If session policy restricts it: DENY

Step 6: Identity-based policy — does the user/role policy allow this?
        → If no Allow found anywhere: DENY (default deny)

Final rule: DENY > ALLOW. If anything says DENY, the answer is DENY.
```

```
              If nothing says ALLOW, the answer is DENY.

Visualized:
EXPLICIT DENY anywhere → ❌ DENIED
  ↓ (no explicit deny)
SCP blocks it → ❌ DENIED
  ↓ (SCP allows)
Permission boundary blocks it → ❌ DENIED
  ↓ (within boundary)
Identity policy OR resource policy allows → ✅ ALLOWED
  ↓ (neither allows)
❌ DENIED (default deny)
```

🧩 IAM Roles — Deep Dive

```
Role assumption flow (STS — Security Token Service):

Application/Service                    AWS STS
│                                       │
├── sts:AssumeRole ──────────────────► │
│   (roleArn, sessionName, duration)   │
│                                       │
│◄── Temporary credentials ────────────│
│   AccessKeyId (starts with ASIA...)  │
│   SecretAccessKey                    │
│   SessionToken (REQUIRED with temp)  │
│   Expiration (default 1 hour)        │

Trust policy (resource-based policy ON the role):
Defines WHO is allowed to assume this role.
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Service": "lambda.amazonaws.com"    ← Lambda can assume this role
    },
    "Action": "sts:AssumeRole"
  }]
}

EC2 instance role trust policy:
{
  "Principal": {"Service": "ec2.amazonaws.com"},
  "Action": "sts:AssumeRole"
}

Cross-account role assumption:
{
  "Principal": {
```

```
    "AWS": "arn:aws:iam::CICD-ACCOUNT-ID:role/DeployerRole"
  },
  "Action": "sts:AssumeRole",
  "Condition": {
    "StringEquals": {
      "sts:ExternalId": "unique-external-id-12345"  ← confused deputy
protection
    }
  }
}

ExternalId = protection against confused deputy attack:
If AttackerCorp tricks your trusted service into assuming your role,
ExternalId (a secret only you and your service know) prevents it.
```

🧩 Least Privilege in Practice

```
Common mistakes and how to fix them:

❌ Wrong: Attach AdministratorAccess to application role
   "It's easier and we'll tighten it later" → never happens

✅ Right: Generate least-privilege policy from CloudTrail
   Step 1: Run application with broad permissions (dev environment)
   Step 2: Use IAM Access Analyzer → Generate Policy from CloudTrail
   Step 3: Review and attach generated minimal policy

❌ Wrong: Wildcard resources
   "s3:*" on "Resource": "*"

✅ Right: Specific resources
   "s3:GetObject" on "arn:aws:s3:::my-app-bucket/*"
   "s3:ListBucket" on "arn:aws:s3:::my-app-bucket"
   (Note: ListBucket requires the bucket ARN, not bucket/*)

❌ Wrong: Shared IAM user credentials for multiple services
   One access key used by 5 different Lambda functions

✅ Right: Separate execution role per Lambda function
   Each Lambda has its own role with only what it needs
   Compromise of one = only that function's permissions exposed

Permission Boundaries — when to use them:
├── You delegate IAM creation to developer teams
│   (without boundary: devs could escalate their own privileges!)
└── Boundary: devs can create roles BUT roles cannot exceed boundary
   {
     "Effect": "Allow",
     "Action": ["s3:*", "lambda:*", "dynamodb:*"],
     "Resource": "*"
```

```
    }
    → Devs can create any role, but that role can only touch
S3/Lambda/DynamoDB
```

🧩 Key IAM CLI Commands

```
# Create IAM role with trust policy
aws iam create-role \
  --role-name LambdaExecutionRole \
  --assume-role-policy-document file://trust-policy.json

# Attach AWS managed policy to role
aws iam attach-role-policy \
  --role-name LambdaExecutionRole \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole

# Create and attach custom inline policy
aws iam put-role-policy \
  --role-name LambdaExecutionRole \
  --policy-name DynamoDBReadAccess \
  --policy-document file://dynamodb-read-policy.json

# Simulate IAM policy (without making actual API calls)
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:role/LambdaExecutionRole \
  --action-names "s3:GetObject" "dynamodb:PutItem" \
  --resource-arns "arn:aws:s3:::my-bucket/*"

# Check who can assume a role
aws iam get-role --role-name LambdaExecutionRole \
  --query 'Role.AssumeRolePolicyDocument'

# List all policies attached to a role
aws iam list-attached-role-policies --role-name LambdaExecutionRole
aws iam list-role-policies --role-name LambdaExecutionRole  # inline
policies

# Assume a role manually (for cross-account access)
aws sts assume-role \
  --role-arn arn:aws:iam::PROD-ACCOUNT:role/DeployRole \
  --role-session-name my-deploy-session \
  --duration-seconds 3600

# Generate least-privilege policy from CloudTrail (IAM Access Analyzer)
aws accessanalyzer start-policy-generation \
  --policy-generation-details
'{"principalArn":"arn:aws:iam::123:role/MyRole"}' \
  --cloud-trail-details '{
    "trails":
```

```
  [{"cloudTrailArn":"arn:aws:cloudtrail:...","allRegions":true}],
      "accessRole": "arn:aws:iam::123:role/AccessAnalyzerRole",
      "startTime": "2024-01-01T00:00:00Z",
      "endTime": "2024-02-01T00:00:00Z"
    }'

  # Rotate access keys (for service accounts — better to use roles)
  aws iam create-access-key --user-name service-account-user
  aws iam update-access-key --access-key-id OLD-KEY-ID --status Inactive
  aws iam delete-access-key --access-key-id OLD-KEY-ID
```

🧩 IAM Access Analyzer

```
IAM Access Analyzer answers: "Who outside my account has access to my
resources?"

Analyzes resource-based policies on:
├── S3 buckets
├── IAM roles
├── KMS keys
├── Lambda functions
├── SQS queues
└── Secrets Manager secrets

Findings:
├── ACTIVE:   external access currently exists → review and archive or fix
├── ARCHIVED: acknowledged as intentional (e.g., cross-account partner
access)
└── RESOLVED: the policy was changed and access no longer exists

Policy validation:
aws accessanalyzer validate-policy \
  --policy-document file://my-policy.json \
  --policy-type IDENTITY_POLICY
# Returns: SUGGESTION, WARNING, ERROR findings
# Catches: unused conditions, redundant statements, typos in action names
```

💬 Short Crisp Interview Answer

"IAM has four identity types: Users for humans or long-lived service credentials (avoid for services), Groups for organizing users, Roles for AWS services and cross-account access using temporary credentials, and IAM Identity Center for federated human SSO. The policy evaluation order is critical: explicit Deny beats everything, then SCPs, then permission boundaries, then identity and resource policies — and the default is always Deny. In production I follow three rules: no long-lived access keys for services (use roles), least privilege generated from actual usage via IAM Access Analyzer, and permission boundaries when delegating IAM management to teams to prevent privilege escalation."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
ListBucket vs GetObject ARN	`s3:ListBucket` needs bucket ARN, `s3:GetObject` needs `bucket/*`. Mixing them up = silent denial
Explicit Deny is absolute	An explicit Deny in ANY policy cannot be overridden by any Allow, even AdministratorAccess
Groups are not principals	You cannot reference a group in a resource-based policy's Principal — only users, roles, accounts
Temporary credentials need SessionToken	ASIA... access keys ALWAYS need the SessionToken — forgetting it causes auth failure
AssumeRole requires both trust + permissions	Trust policy allows assumption; identity policy allows the actual actions. Both required
Permission boundaries don't grant	A boundary saying "Allow s3:*" doesn't grant s3 access — identity policy must also grant it
IAM is global	IAM users, roles, and policies are global — not per-region. One role works everywhere
Policy character limit	Inline policies: 2,048 chars. Managed policies: 6,144 chars. Role trust: 4,096 chars

---

1.3 AWS CLI & SDK Basics

🟢 What It Is in Simple Terms

The AWS CLI and SDKs are how you interact with AWS programmatically — instead of clicking in the Console. The CLI is a command-line tool for humans and scripts. The SDKs are libraries in Python, JavaScript, Go, Java, etc. for building applications. Both call the same underlying AWS REST APIs.

🧩 CLI Installation and Configuration

```
# Install AWS CLI v2 (Linux)
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip && sudo ./aws/install
aws --version  # aws-cli/2.x.x

# Configure default profile
aws configure
# AWS Access Key ID:     AKIAIOSFODNN7EXAMPLE
# AWS Secret Access Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

```
# Default region:        us-east-1
# Default output format: json   ← also: text, table, yaml

# This creates:
# ~/.aws/credentials  ← access key ID and secret key
# ~/.aws/config       ← region, output format, profiles
```

🧩 Named Profiles — The Right Way to Manage Multiple Accounts

```
# ~/.aws/config
[default]
region = us-east-1
output = json

[profile dev]
region = us-east-1
role_arn = arn:aws:iam::DEV-ACCOUNT:role/DevRole
source_profile = default

[profile prod]
region = us-east-1
role_arn = arn:aws:iam::PROD-ACCOUNT:role/ReadOnlyRole
source_profile = default
mfa_serial = arn:aws:iam::ROOT-ACCOUNT:mfa/my-yubikey

# Use a specific profile
aws s3 ls --profile prod
AWS_PROFILE=prod aws ec2 describe-instances

# ~/.aws/credentials (for the source profile with actual keys)
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI...

# Credential precedence (highest → lowest):
# 1. CLI flags (--region, --profile)
# 2. Environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY,
AWS_SESSION_TOKEN)
# 3. AWS config file profiles
# 4. EC2/ECS/Lambda instance metadata (IMDS) — for roles on instances
# 5. Container credentials (ECS task role)
```

🧩 CLI Output Formats and Filtering

```
# Output formats: json (default), text, table, yaml
aws ec2 describe-instances --output table
aws ec2 describe-instances --output yaml
```

```
# --query: JMESPath expressions to filter JSON output
# Get all instance IDs in us-east-1
aws ec2 describe-instances \
  --query 'Reservations[*].Instances[*].InstanceId' \
  --output text

# Get running instances with their private IPs and names
aws ec2 describe-instances \
  --filters "Name=instance-state-name,Values=running" \
  --query 'Reservations[*].Instances[*].
{ID:InstanceId,IP:PrivateIpAddress,Name:Tags[?Key==`Name`]|[0].Value}' \
  --output table

# Get specific fields from S3 bucket list
aws s3api list-buckets \
  --query 'Buckets[?starts_with(Name, `prod-`)].
{Name:Name,Created:CreationDate}' \
  --output json

# Pagination — handle large result sets
aws ec2 describe-instances \
  --max-items 50 \
  --page-size 50
# Or use --no-paginate to get ALL results (careful with huge datasets)

# Filter by tag in CLI
aws ec2 describe-instances \
  --filters \
    "Name=tag:Environment,Values=prod" \
    "Name=tag:Team,Values=platform" \
    "Name=instance-state-name,Values=running"
```

🧩 SDK Usage — Python (boto3) Patterns

```
import boto3
from botocore.exceptions import ClientError

# Session management — equivalent to profiles
session = boto3.Session(
    profile_name='prod',        # uses ~/.aws/config [profile prod]
    region_name='us-east-1'
)

# Service clients vs resources
ec2_client = session.client('ec2')     # low-level, maps 1:1 to API
ec2_resource = session.resource('ec2') # higher-level, ORM-like

# Assume role within code (cross-account)
def get_cross_account_session(role_arn: str, session_name: str):
```

```
    sts = boto3.client('sts')
    creds = sts.assume_role(
        RoleArn=role_arn,
        RoleSessionName=session_name,
        DurationSeconds=3600
    )['Credentials']

    return boto3.Session(
        aws_access_key_id=creds['AccessKeyId'],
        aws_secret_access_key=creds['SecretAccessKey'],
        aws_session_token=creds['SessionToken'],   # ← critical, never
forget
        region_name='us-east-1'
    )

prod_session = get_cross_account_session(
    'arn:aws:iam::PROD-ACCOUNT:role/ReadRole',
    'my-automation'
)
prod_s3 = prod_session.client('s3')

# Proper error handling with ClientError
def get_secret(secret_name: str) -> str:
    client = boto3.client('secretsmanager')
    try:
        response = client.get_secret_value(SecretId=secret_name)
        return response['SecretString']
    except ClientError as e:
        error_code = e.response['Error']['Code']
        if error_code == 'ResourceNotFoundException':
            raise ValueError(f"Secret {secret_name} not found")
        elif error_code == 'AccessDeniedException':
            raise PermissionError(f"No access to secret {secret_name}")
        else:
            raise  # re-raise unexpected errors

# Pagination with paginator (critical for large datasets)
def list_all_instances(region: str) -> list:
    ec2 = boto3.client('ec2', region_name=region)
    paginator = ec2.get_paginator('describe_instances')

    instances = []
    for page in paginator.paginate(
        Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]
    ):
        for reservation in page['Reservations']:
            instances.extend(reservation['Instances'])
    return instances

# Waiter — poll until resource reaches desired state
def wait_for_instance(instance_id: str):
    ec2 = boto3.client('ec2')
    waiter = ec2.get_waiter('instance_running')
    waiter.wait(
```

```
        InstanceIds=[instance_id],
        WaiterConfig={'Delay': 15, 'MaxAttempts': 20}
    )
    print(f"Instance {instance_id} is running")
```

🧩 Environment Variables for CI/CD

```
# Environment variables override config files — use in CI/CD pipelines
export AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI..."
export AWS_SESSION_TOKEN="AQoXnyc4Pi..."    # required for assumed roles
export AWS_DEFAULT_REGION="us-east-1"
export AWS_DEFAULT_OUTPUT="json"

# In GitHub Actions — inject from secrets
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123:role/GitHubActionsRole
    aws-region: us-east-1
# Uses OIDC — no stored access keys at all!

# Check current identity (always run this first in debugging)
aws sts get-caller-identity
# Returns:
# {
#   "UserId": "AROAIOSFODNN7EXAMPLE:my-session",
#   "Account": "123456789012",
#   "Arn": "arn:aws:sts::123456789012:assumed-role/MyRole/my-session"
# }
```

🧩 CLI Power User Techniques

```
# Run command across ALL regions
for region in $(aws ec2 describe-regions --query 'Regions[].RegionName' --
output text); do
  echo "=== $region ==="
  aws ec2 describe-instances \
    --region $region \
    --filters "Name=instance-state-name,Values=running" \
    --query 'Reservations[*].Instances[*].
[InstanceId,InstanceType,PrivateIpAddress]' \
    --output text
done

# Find all unencrypted EBS volumes (security audit)
aws ec2 describe-volumes \
```

```
  --filters "Name=encrypted,Values=false" \
  --query 'Volumes[*].
{ID:VolumeId,Size:Size,AZ:AvailabilityZone,State:State}' \
  --output table

# Get total cost for last month via CLI
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --query 'ResultsByTime[0].Total.BlendedCost.Amount' \
  --output text

# Tail CloudWatch Logs like 'tail -f'
aws logs tail /aws/lambda/my-function --follow

# Copy S3 objects between accounts (needs permissions on both sides)
aws s3 sync s3://source-bucket/ s3://dest-bucket/ \
  --source-region us-east-1 \
  --region eu-west-1

# Dry run (check permissions without actually doing it)
aws ec2 run-instances \
  --image-id ami-0abc123 \
  --instance-type t3.micro \
  --dry-run
# Returns DryRunOperation error if you HAVE permission (confusing but
correct)
# Returns UnauthorizedOperation if you DON'T have permission
```

💬 Short Crisp Interview Answer

"The CLI and SDKs both call the same AWS REST APIs under the hood — the CLI is for scripts and humans, SDKs for application code. Credentials follow a strict precedence: CLI flags → environment variables → config file → instance metadata (IMDS). In production I never use long-lived access keys in CI/CD — instead I use IAM OIDC with GitHub Actions or CodeBuild roles that assume cross-account roles with temporary STS credentials. Three patterns I always use: named profiles for multi-account work, paginators in boto3 for any list operation (truncation bites you at scale), and sts:get-caller-identity as the first debug step when anything auth-related fails."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Credential precedence	Env vars override config files. A stale `AWS_ACCESS_KEY_ID` env var causes auth failures even with correct config
SessionToken required	Temporary creds (from AssumeRole) require `AWS_SESSION_TOKEN` — forgetting it causes `InvalidClientTokenId`

Gotcha	Detail
--dry-run is backwards	DryRunOperation = you HAVE permission. UnauthorizedOperation = you DON'T
Pagination is not automatic	Without paginators, `describe_instances` returns max 1,000 results silently truncated
SDK regional clients	`boto3.client('s3')` with no region uses default. But some services require explicit region
CLI query JMESPath	`--query` filters happen client-side — all data fetched from API first then filtered
Profile vs environment vars	`AWS_PROFILE` env var overrides `--profile` flag when both are set

1.4 AWS Organizations & Multi-account Strategy

🟢 What It Is in Simple Terms

AWS Organizations lets you manage multiple AWS accounts from a central management account. Instead of one giant account where everything lives together, multi-account strategy puts different workloads, environments, and teams in separate accounts — providing isolation, independent billing, and enforceable security guardrails at the organizational level.

🔍 Why Multi-account Exists / What Problem It Solves

```
Single-account problems:
├── Blast radius: one misconfiguration can affect everything
├── IAM complexity: 50 teams sharing one account = IAM spaghetti
├── No hard isolation: dev workloads compete with prod for service limits
├── Cost visibility: one bill for everything → impossible to chargeback
└── Security: one compromised credential = entire company exposed

Multi-account benefits:
├── Security boundary: IAM cannot cross account boundaries by default
├── Blast radius control: an incident in dev doesn't touch prod
├── Service limit isolation: dev hitting EC2 limit doesn't affect prod
├── Cost transparency: per-account billing → team/product chargeback
├── Compliance: PCI/HIPAA scope reduced to specific accounts
└── Innovation: sandbox accounts let teams experiment safely
```

🧩 AWS Organizations — Structure

```
AWS Organizations hierarchy:

Root
├── Management Account (root of the org — formerly called master account)
│   Used ONLY for: billing, organizational management, Control Tower
│   Never run production workloads here
│
├── Organizational Unit (OU): Security
│   ├── Log Archive Account    ← all CloudTrail, Config logs centralized
here
│   └── Audit/Security Account ← security team has read access to all
accounts
│
├── Organizational Unit (OU): Infrastructure
│   ├── Network Account        ← Transit Gateway, DNS, Direct Connect hub
│   └── Shared Services Account← CI/CD, container registries, artifact
repos
│
├── Organizational Unit (OU): Workloads
│   ├── OU: Production
│   │   ├── prod-payments account
│   │   ├── prod-orders account
│   │   └── prod-platform account
│   └── OU: Non-Production
│       ├── staging account
│       ├── dev account (shared dev environment)
│       └── sandbox accounts (individual developer playgrounds)
│
└── Organizational Unit (OU): Suspended
    └── Decommissioned accounts (closed but retained for billing history)

Key concept: OUs can be nested. Apply policies at OU level → inherited by
all accounts.
```

🧩 Service Control Policies (SCPs)

```
SCPs = maximum permission guardrails applied at the OU or account level
       Even the ROOT USER in an account cannot exceed what the SCP allows

SCPs restrict what IAM permissions can DO — they don't grant permissions.
An SCP Allow + IAM Allow = Allowed
An SCP Deny + IAM Allow = DENIED (SCP wins)
No SCP Allow + IAM Allow = DENIED (SCP must explicitly allow OR no SCP
applied)

Default SCP: FullAWSAccess (allows everything) — applied to root by
default

Common production SCPs:
```

```
1. Prevent disabling security services:
{
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging",
    "cloudtrail:DeleteTrail",
    "guardduty:DeleteDetector",
    "config:DeleteConfigurationRecorder"
  ],
  "Resource": "*"
}

2. Restrict to approved regions only:
{
  "Effect": "Deny",
  "NotAction": [
    "iam:*",           ← global service — must exclude
    "organizations:*", ← global service
    "route53:*",       ← global service
    "cloudfront:*",    ← global service
    "sts:*"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": ["us-east-1", "us-east-2", "eu-west-1"]
    }
  }
}

3. Deny root user access (enforce MFA + break-glass procedures):
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringLike": {
      "aws:PrincipalArn": ["arn:aws:iam::*:root"]
    }
  }
}

4. Require encryption on S3 objects:
{
  "Effect": "Deny",
  "Action": "s3:PutObject",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": ["aws:kms", "AES256"]
    }
  }
}
```

```
# Create an SCP
aws organizations create-policy \
  --content file://deny-non-approved-regions.json \
  --name "DenyNonApprovedRegions" \
  --type SERVICE_CONTROL_POLICY \
  --description "Restrict actions to approved regions only"

# Attach SCP to an OU
aws organizations attach-policy \
  --policy-id p-abc123 \
  --target-id ou-root-xxxxxxxx

# List SCPs on an account
aws organizations list-policies-for-target \
  --target-id 123456789012 \
  --filter SERVICE_CONTROL_POLICY

# Simulate effective permissions (SCP + IAM combined)
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::ACCOUNT:role/DevRole \
  --action-names "ec2:RunInstances" \
  --resource-arns "*" \
  --caller-arn arn:aws:iam::ACCOUNT:role/DevRole
```

🧩 Cross-Account Access Patterns

```
Pattern 1: Role assumption (most common)

Developer in "jump account" → assumes role in target account

# In target account: create role that trusts the source account
Trust policy on TargetAccountRole:
{
  "Principal": {
    "AWS": "arn:aws:iam::SOURCE-ACCOUNT-ID:root"  ← entire source account
  },
  "Action": "sts:AssumeRole"
}

# More specific trust (only specific role in source account):
{
  "Principal": {
    "AWS": "arn:aws:iam::CICD-ACCOUNT:role/DeployerRole"
  },
  "Action": "sts:AssumeRole"
}
```

```
Developer workflow:
aws sts assume-role \
  --role-arn arn:aws:iam::PROD-ACCOUNT:role/ReadOnlyRole \
  --role-session-name dev-john-prod-access

Pattern 2: AWS Config file with role chaining
[profile prod-readonly]
role_arn = arn:aws:iam::PROD-ACCOUNT:role/ReadOnlyRole
source_profile = default  ← assume role using credentials from [default]

aws s3 ls --profile prod-readonly
# CLI automatically calls sts:AssumeRole → uses temp creds → calls
s3:ListBuckets

Pattern 3: Resource-based policies (S3, KMS, SQS cross-account)
# S3 bucket in Account B — allow Account A to read
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::ACCOUNT-A:role/ReaderRole"
  },
  "Action": ["s3:GetObject", "s3:ListBucket"],
  "Resource": [
    "arn:aws:s3:::shared-data-bucket",
    "arn:aws:s3:::shared-data-bucket/*"
  ]
}
# Account A's ReaderRole must ALSO have IAM permission to call
s3:GetObject
# Both sides must allow for cross-account access
```

Real-World Login Flow (Modern Teams)

Most companies today use **IAM Identity Center (SSO)** federated from Okta / Azure AD / Google Workspace:

1. IT shares SSO portal URL on day one (e.g., company.awsapps.com/start)
2. Engineer logs in via corporate credentials — no IAM user created in AWS
3. MFA is enforced at the **IdP layer** (Okta/Azure AD), not inside AWS
4. IAM Identity Center issues temporary STS credentials per session
5. No long-lived access keys ever created

CLI setup:

```
aws configure sso
aws sso login --profile prod-readonly
```

> ⚠ You never create your own IAM user in a production account. SSO + role assumption is the correct, auditable, revocable pattern.

🧩 Account Vending Machine — Account Factory

```
Problem: Creating accounts manually doesn't scale.
         "I need a new dev environment for the payments team" → takes a
week.

Solution: Automated account provisioning (Account Vending Machine /
Account Factory)

AWS Control Tower Account Factory:
├── Self-service portal (or Terraform/API)
├── New account request → automatically:
│   ├── Creates AWS account in the right OU
│   ├── Applies all baseline SCPs
│   ├── Enables CloudTrail, AWS Config, GuardDuty
│   ├── Sets up IAM Identity Center access
│   ├── Provisions baseline VPC or no-VPC template
│   └── Sends welcome email to account owner
├── Standard account: ~15-30 minutes automated
└── Manual account: 1-2 weeks of tickets

Account Factory for Terraform (AFT):
├── Define account spec in Terraform → PR merge → account created
└── All customizations versioned in git

Baseline guardrails applied to every new account:
├── CloudTrail → centralized to log archive account
├── AWS Config → enabled, rules for compliance
├── GuardDuty → member account under central detector
├── Security Hub → findings aggregated centrally
└── Default VPC deleted (create custom VPCs with your standards)
```

🧩 Consolidated Billing

```
Consolidated billing = all accounts' charges combined on management
account bill
                       with sharing of volume discounts and Savings Plans

Benefits:
├── Volume discounts: 50 accounts' S3 storage pooled → hit higher discount
tiers
├── Reserved Instance sharing: unused RIs in Account A apply to Account B
├── Savings Plans sharing: compute SP purchased in one account → all
accounts
└── Single invoice: one payment, one vendor relationship

Cost allocation:
├── Per-account costs visible in Cost Explorer
├── Charge back to teams using AWS Cost Categories or tags
```

```
└── Budget per account → alerts to account owners

# Enable cost allocation tags across organization
aws organizations enable-aws-service-access \
  --service-principal member.org.stacksets.cloudformation.amazonaws.com

# List accounts in org for automation
aws organizations list-accounts \
  --query 'Accounts[?Status==`ACTIVE`].{ID:Id,Name:Name,Email:Email}' \
  --output table
```

🧩 Common Multi-Account Architectures

```
Architecture 1: Basic 3-account setup (small company)
├── Management:  billing only, org management
├── Shared:      CI/CD, DNS, shared services
└── Production:  all prod workloads
                 (simple but limited isolation)

Architecture 2: Environment-per-account (most common)
├── Management:  billing only
├── Security:    log archive + audit
├── Shared:      CI/CD, ECR, Artifactory
├── Dev:         shared dev environment
├── Staging:     pre-prod testing
└── Production:  production workloads
                 (good isolation, manageable complexity)

Architecture 3: Product-per-account (large org, strict isolation)
├── Management:    billing only
├── Security:      log archive + audit
├── Platform:      network hub, shared services
├── payments-dev / payments-staging / payments-prod
├── orders-dev   / orders-staging   / orders-prod
└── data-dev     / data-staging     / data-prod
                  (maximum isolation, higher overhead)

Choosing an architecture:
├── < 5 engineers:   single account with good IAM is fine
├── 5-50 engineers:  environment-per-account
└── 50+ engineers:   product + environment per account
```

💬 Short Crisp Interview Answer

"AWS Organizations provides the management plane for multi-account strategy. The core value of multiple accounts is security isolation — IAM cannot cross account boundaries by default, so a compromised credential in dev cannot touch prod. I structure accounts around environments (prod, staging, dev) and

function (security/logging, networking, shared services), with SCPs at the OU level enforcing non-negotiable guardrails: prevent disabling CloudTrail, restrict to approved regions, deny root user usage. Cross-account access uses role assumption — the target account's role has a trust policy allowing the source account's role, and the source side needs IAM permission to call sts:AssumeRole. Consolidated billing pools volume discounts and Savings Plans across all accounts. For account provisioning, Control Tower Account Factory automates the baseline — CloudTrail, GuardDuty, Security Hub — so a new account is secure by default in 30 minutes instead of days."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Management account has no SCP restrictions	SCPs don't apply to the management account — never run workloads there
SCP Deny beats everything	Including the root user. A bad SCP on prod OU can lock out everyone — test carefully
Role assumption limit	Roles can be chained up to 5 hops but session duration decreases with each hop
Leaving an OU doesn't remove SCP effects	Moving account to different OU immediately changes SCP inheritance
Consolidated billing RI sharing opt-out	Accounts can opt out of RI sharing — sometimes needed for cost accuracy per team
Organization CloudTrail only in management	The organization trail must be created from the management account
Account closure takes 90 days	Closed accounts are suspended then permanently deleted after 90 days — data gone
Service limit per account	Each account has its own service limits (vCPU, Lambda concurrency, etc.) — good and bad

---

🔗 Category 1 — Full Connections Map

```
AWS CORE FUNDAMENTALS connects to:

Global Infrastructure
├── VPC                 → deployed within a Region, subnets span AZs
├── Route 53            → latency-based routing, health checks across
AZs/regions
├── CloudFront          → edge locations for CDN, Lambda@Edge
├── ELB                 → deployed across AZs, cross-AZ load balancing
├── RDS Multi-AZ        → primary + standby in separate AZs
└── S3                  → global namespace but data stored in specific
```

```
region

IAM
├── Every AWS service  → all API calls evaluated through IAM
├── CloudTrail         → logs every IAM action with principal identity
├── KMS                → IAM controls who can use which keys
├── Organizations/SCPs → restrict what IAM policies can grant at org level
├── Cognito            → IAM roles for federated identity (web/mobile
users)
└── IAM Access Analyzer→ validates policies, finds external access

CLI / SDK
├── All AWS services   → every service has CLI commands and SDK methods
├── CloudShell         → browser-based CLI environment (no install needed)
├── IAM                → credentials used by CLI/SDK evaluated through IAM
├── STS                → SDK calls AssumeRole to get temporary credentials
└── Lambda             → SDK used within Lambda to call other AWS services

Organizations / Multi-account
├── IAM Identity Center→ SSO across all org accounts
├── Control Tower      → governance + account factory on top of
Organizations
├── CloudTrail         → organization trail centralizes logs from all
accounts
├── GuardDuty          → organization-level threat detection
├── Security Hub       → aggregates findings across all org accounts
├── AWS Config         → conformance packs across org
└── RAM                → Resource Access Manager shares resources across
accounts
```

📌 Quick Reference — Category 1 Interview Cheat Sheet

Topic	Key Fact
Region isolation	Regions are fully isolated — no automatic cross-region data flow
AZ count	Minimum 3 AZs for production. us-east-1 has 6 AZs
AZ name mapping	AZ names are account-specific. Use AZ IDs (use1-az1) for cross-account coordination
Inter-AZ cost	$0.01/GB data transfer between AZs in same region
Edge locations	400+ globally for CloudFront, Route 53, WAF
IAM evaluation order	Explicit Deny → SCP → Resource policy → Permission Boundary → Identity policy → Default Deny
IAM Groups	Not a principal — cannot appear in resource-based policy Principal field
Temp credentials	Always require SessionToken alongside AccessKeyId + SecretAccessKey

Topic	Key Fact
Permission boundaries	Don't grant permissions — only cap what identity policy can grant
ExternalId	Protects against confused deputy attack in cross-account role assumption
CLI credential precedence	Flags → Env vars → Config file → IMDS → Container → Process
Paginator requirement	Any list/describe operation may be paginated — always use paginators in code
dry-run behavior	DryRunOperation = you have permission. UnauthorizedOperation = you don't
Multi-account isolation	IAM cannot cross account boundaries by default — strongest isolation primitive
SCP scope	SCPs don't apply to management account. Always apply to child accounts via OUs
Consolidated billing	Pools RI/SP discounts. Each account still has independent service limits
Account Factory	Provisions secure-by-default accounts in 30 minutes (vs days manually)
Cross-account access	Trust policy on role (who can assume) + identity policy (what they can do) — both required

Category 1: AWS Core Fundamentals — Complete Interview Guide